MedHire Candidate Backend Handoff

Prepared: 9 October 2026 **Status:** Frontend integration contract; backend routes and response shapes require confirmation before implementation.

Purpose and scope

This document describes the candidate portal API expected by the MedHire frontend: shared authentication, nine-step profile onboarding, candidate dashboard and profile, resumes, job discovery and applications, saved jobs, privacy settings, and account export/deletion. The candidate and recruiter frontends are in `medhire-main`; the separate admin portal is in `adminportal2026`.

Frontend browser routes (for example `/candidate/profile/personal`) are not API routes. Candidate-owned API calls identify the candidate from the bearer token; the client must not send a candidate ID as an ownership claim. All resource routes below are **proposed** and must be reconciled with the backend's deployed OpenAPI contract.

Environments and shared conventions

- Frontend API base defaults to `/api` (`VITE_API_BASE_URL`). Resource paths below are relative to that base, e.g. `GET /candidates/me/profile` resolves to `/api/candidates/me/profile`.
- Send `Authorization: Bearer <accessToken>` for protected calls and JSON for structured bodies. Resume/photo uploads use `multipart/form-data`; let the browser set the multipart boundary.
- Use ISO 8601 UTC timestamps for instants and `YYYY-MM-DD` for calendar dates. Use stable opaque IDs.
- Agree one success envelope, pagination format, validation/error envelope, status codes and idempotency policy across candidate, recruiter and admin APIs. Frontend readers currently accept a few common response envelopes, but a canonical schema is needed.
- Do not return demo or other users' data on empty/error responses. A 204 response is accepted for delete operations.

Shared authentication

Candidate registration and login use the shared auth contract, not candidate profile endpoints. The archived auth handoff PDF is a visual reference; confirm it against the current shared auth implementation.

Method and path	Purpose / request	Required behavior
`POST /auth/register`	Candidate registration; role `candidate`, full name, email, phone and password	Validate and normalize input; create unverified account; never return password/hash.
`POST /auth/login`	Email, password, optional remember-me	Return access session and user identity/role; candidate UI routes by server role.
`POST /auth/verify-code`	`{ "email": "...", "code": "...", "purpose": "registration" }`	This is the endpoint the candidate and recruiter frontends call. Expire and consume the code; throttle attempts.

Method and path	Purpose / request	Required behavior
`POST /auth/resend-code`	`{ "email": "...", "purpose": "registration" }`	Resend registration code with throttling and generic responses.
`POST /auth/forgot-password`	Email	Generic response whether account exists; rate-limit and send short-lived code.
`POST /auth/verify-code`	Email, code, purpose `password_reset`	Validate and issue single-use reset grant. The same route also verifies `email_change` codes.
`POST /auth/reset-password`	Email, reset token, new password	Atomically consume grant, update password, revoke affected sessions.
`POST /auth/change-password`	`{ "currentPassword": "...", "newPassword": "..." }`	Verify current password, enforce password rules, and revoke other sessions as agreed.
`POST /auth/change-email`	`{ "newEmail": "...", "password": "..." }`	Verify current password and send an `email_change` code; change the address only after verification.
`GET /auth/me`	Authenticated request	Return the current server-verified user and recruiter organization status.
`POST /auth/refresh`, `POST /auth/logout`	Refresh session / revoke session	Rotate refresh token with reuse detection; logout revokes server-side session. Confirm cookie vs bearer strategy and CSRF/CORS policy.

Enforce password policy and role on the server. Do not reveal account existence during recovery. Candidate identity, email verification and session status are server-owned.

Onboarding and profile APIs

The UI saves a valid step when the candidate selects Next. Progress must persist across devices/sessions. Skipping an optional step must never replace saved section data with an empty payload. The final completion action is distinct from saving preferences.

Step / purpose	Method and path	Request
Read profile / progress	`GET /candidates/me/profile`; `GET /candidates/me/onboarding`	None
Resume upload (optional)	`POST /candidates/me/resume`	Multipart key `resume`; frontend accepts PDF/DOC/DOCX up to 5 MB. Confirm backend limit, MIME inspection, malware scanning and storage policy.
Personal information	`PUT /candidates/me/personal`	`firstName`, `lastName`, `phoneNumber`, `dateOfBirth`, `gender`, `country`, `address`, `city`, `state`, `zipCode`. Email is auth-owned/read-only and omitted. Gender is lower-case; country uses `US`, `IN`, `CA`, `GB`, `AU`, or `OTHER`; date is `YYYY-MM-DD`.

Step / purpose	Method and path	Request
Professional information	`PUT /candidates/me/professional`	`currentJobTitle`, `yearsOfExperience`, `currentEmployer`, `specialty`, `licenseNumber`, `licenseState`, `npiNumber`, `professionalSummary`
Education	`PUT /candidates/me/education`	`education[]`: `id`, `degree`, `fieldOfStudy`, `institutionName`, `graduationYear`
Work experience	`PUT /candidates/me/experience`	`workExperience[]`: `id`, `jobTitle`, `employerName`, `employmentType`, `location`, `startMonth`, `startYear`, `endMonth`, `endYear`, `currentlyWorking`, `description`
Skills	`PUT /candidates/me/skills`	`skills[]` of `{ name }`
Certifications	`PUT /candidates/me/certifications`	`certifications[]`: `id`, `certificationName`, `issuingOrganization`, `credentialId`, `issueDate`, `expiryDate`, `doesNotExpire`
Career preferences	`PUT /candidates/me/preferences`	`desiredJobTitle`, `preferredSpecialties[]`, `employmentTypes[]`, `preferredLocations[]`, `workSetting`, `shiftPreference`, `expectedSalary`, `salaryType`, `availableFrom`, `willingToRelocate`, `remotePreference`
Optional progress/skip	`PATCH /candidates/me/onboarding/steps/{stepKey}`	`{ "status": "SKIPPED" }`; step keys: `resume`, `personal`, `professional`, `education`, `experience`, `skills`, `certifications`, `preferences`.
Complete onboarding	`POST /candidates/me/profile/complete`	`{}`; validate required fields and agreed skip policy, then return durable completion state.
Update profile	`PATCH /candidates/me/profile`	Editable profile fields only; preserve auth-owned email and server-owned status/strength.
Upload profile photo	`POST /candidates/me/profile/photo`	Proposed endpoint, multipart key `photo`; confirm route, MIME/size limits and access policy.

Progress statuses: `NOT_STARTED`, `IN_PROGRESS`, `COMPLETED`, `SKIPPED`. Later valid data may move a skipped step to completed. Confirm whether repeatable arrays replace the full section or support item-level upserts, and define stable item IDs and conflict behavior.

Required completion policy currently represented by the frontend: personal details, professional title/experience/specialty, education, skills and career preferences. Resume, work history and certifications may be skipped; candidates with over one year of experience need valid work history for full profile strength. Optional NPI, when present, is ten digits. Salary is USD and requires a period when supplied. Backend validation remains authoritative.

Return field-level errors in a stable envelope, for example:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "fields": { "phoneNumber": "Enter a valid phone number." },
    "requestId": "request-id"
  }
}
```

Candidate dashboard and self-service APIs

All `/candidates/me/...` records must be scoped to the authenticated candidate. Empty or unavailable resources render empty/error states; never substitute fixture identities or activity.

Purpose	Method and path	Expected request / response
Dashboard	`GET /candidates/me/dashboard`	Metrics such as `profileViews`, `applications`, `savedJobs`, `profileStrength`, `recommendedJobs`, `recentActivity`
Summary analytics	`GET /candidates/me/summary`	Candidate-owned aggregates and `chartSeries`
Full profile	`GET /candidates/me/profile`	Candidate's profile and completion data
Applications	`GET /candidates/me/applications`	Candidate's applications; agree canonical list envelope and status enum
Saved jobs	`GET /candidates/me/saved-jobs`	Saved job IDs and/or embedded job records
Save / remove saved job	`POST /candidates/me/saved-jobs` with `{ "jobId": "..." }`; `DELETE /candidates/me/saved-jobs/{jobId}`	Idempotent save behavior; delete only caller's bookmark
Resume list	`GET /candidates/me/resumes`	IDs, filename, MIME type, size, upload time, review status, active flag
Resume upload/delete	`POST /candidates/me/resume`; `DELETE /candidates/me/resumes/{resumeId}`	Upload multipart key `resume`; authorize ownership, validate actual file content and size.
Resume download	`GET /candidates/me/resumes/{resumeId}/download`	Authorized stream or short-lived private URL; never expose public resume URLs.
Privacy	`GET /candidates/me/privacy`; `PUT /candidates/me/privacy`	`profileVisible`, `recruiterDiscoverable`, `resumeVisible`, `contactVisible`, `openToWork`; confirm supported fields and recruiter masking semantics.
Export personal data	`GET /candidates/me/data-export`	Proposed by current adapter; return authorized export or short-lived download URL and audit the export.

Purpose	Method and path	Expected request / response
Delete account	`DELETE /candidates/me/account`	Proposed by current adapter; body `{ "password": "...", "confirmation": "DELETE" }`; define re-authentication, retention, anonymization and reversible window.

Job discovery and applications

Purpose	Method and path	Query / body
Search jobs	`GET /jobs`	`q`, `location`, `specialty`, `employmentType`, `workSetting`, `page`, `pageSize`, `sort`; frontend may send company as `hospital`—confirm canonical filter.
Job detail	`GET /jobs/{jobId}`	Return only searchable/active jobs and their requirements, location, employment type and organization summary.
Apply	`POST /jobs/{jobId}/applications`	`{ "resumeId": "...", "coverLetter": "..." }`; enforce active job, approved resume, no duplicate application, candidate ownership and cover letter limit (3,000 characters).

Search is debounced in the UI. List records should expose stable ID, title, organization, location, employment type and specialty; define a single paginated response envelope. The dashboard supports save/apply actions from recommendations too.

Security and backend decisions

- Derive candidate identity only from the verified access token; enforce ownership on every profile, application, saved-job, resume, photo, export and delete route.
- Validate files by content and size, quarantine/scan uploads, store privately, and return authorized streaming or expiring links. Define resume approval states; only an approved resume may be used to apply.
- Do not trust client validation, onboarding progress, profile strength, privacy state, approval state, IDs, or role claims. Recalculate authoritative state on the server.
- Use idempotency/concurrency protection for application submission, deletes, export requests and any monetary/credit side effects. Keep audit logs for sensitive actions without including passwords, tokens or unnecessary PII.
- Confirm all proposed routes and DTOs against the existing backend, including experience representation (UI labels currently include ranges), profile strength formula, response envelopes, job filter names, photo endpoint, logout/revocation, deletion/export semantics, file size/type rules, privacy fields, and status transitions.
- Backend team deliverables: OpenAPI or equivalent, staging URL and CORS/session configuration, role-specific test accounts, representative records/files, canonical enums/errors/pagination, and end-to-end acceptance criteria.

Frontend source map

Candidate APIs live in `src/lib/candidateProfileApi.js` and `src/lib/candidateDashboardApi.js`; validation and data mapping are in `src/lib/candidateProfileValidation.js`, `src/lib/candidateDashboardValidation.js`, `src/lib/candidateDashboardState.js`, and `src/lib/candidateProfileCompletion.js`. Onboarding and dashboard screens are under `src/pages/candidate/`. The frontend contract is proposed until verified against backend responses.